

Protocol-Governed Computing: Realizing the Normative Platform and Its Governed Transformation

© 2026 Bhash Ganti. All rights reserved.

Bhash Ganti (aka Bachi)

Contact: bachipeachy@gmail.com

ORCID Profile: <https://orcid.org/0009-0007-3810-6520>

Companion papers

Part 1 — *Protocol-Governed Computing: An Architecture for Deterministic Declarative Execution*. <https://doi.org/10.5281/zenodo.21879516>

Part 2 — *Protocol-Governed Computing: An Architecture for Closed-Loop Governed Transformation*. <https://doi.org/10.5281/zenodo.21879948>

Abstract

Two companion papers establish the architecture of protocol-governed computing. One defines execution as declarative traversal of a compiled protocol; the other defines evolution as the governed transformation of one executable baseline into the next. Both state what must be true. Neither states what it takes to make it true, nor how one would know it had been achieved.

This paper is about realization. Protocol-Governed Computing (PGC) is realized when both the platform and the transformation of that platform are governed, inspectable, evidence-producing computational objects — the platform as a noun, its transformation as a verb, neither sufficient alone.

Part I develops the **Profiled Normative Platform**: a composition of a governance surface, conformance workloads, and optional business domains under a conformance profile, sealed into an immutable snapshot, reachable across a governed boundary, able to answer questions about itself, and producing evidence of what it did. A platform is a composition under a profile and never a repository. The paper develops the consequences of that distinction and states the conditions under which a platform may be called functioning as checkable properties rather than assertions.

Part II develops **governed transformation** and rests on a proposition the architecture could state but not demonstrate: the current baseline is not context for a change but a *party to it* — supplying reusable structure, bounding admissibility, and serving as the authoritative reference against which every *empirical* claim about the predecessor is verified. Normative claims about the successor — whether the proposed change is

admissible at all — remain governed by the applicable protocol and conformance profile. The transformation function is given formal treatment; the conditions under which it is defined are established stage by stage; and the distinction that makes governed evolution auditable rather than merely careful is drawn: the function is **partial** — not every problem yields a next baseline — while the **process is total**, every failure producing a stated finding at a named stage and never an undefined outcome. Three functions compose into a single account of a governed system's life: transformation changes it, compilation seals it, execution realizes it.

Part III reports what realization surfaced and architectural review did not. Constructing the platform exposed whole capabilities that were unreachable the first time anyone used them, a class of change the governed lifecycle could not express, and rules a domain declared that no artifact enforced. Every one lived on a path that had never been executed, which makes defect discovery a coverage property rather than a maturity one. Architecture establishes invariants; realization establishes whether an implementation instantiates them. These are different epistemic activities, and treating them as one is a plausible explanation for why architectures are so often correct while the systems built from them are not.

A reference implementation is cited throughout as evidence and never as the subject. Each section separates the obligation the architecture imposes from the answer this implementation gives and from the choices incidental to it, so that the requirements remain legible when the implementation does not.

In plain terms

Here is the idea before the argument follows. This section says “change” where the rest of the paper says “transformation” — the same thing, in the more ordinary word. Everywhere after this one, the precise term is meant precisely.

The two papers before this one describe an architecture: how software should execute, and how it should change. Both are arguments about what must be true. Neither tells you what it takes to actually build such a thing, or how you would know you had succeeded rather than merely believed you had. That is what this paper is for.

The claim it makes is that an architecture being correct tells you almost nothing about whether a system built from it is correct — and that finding out is a different kind of work, not a later phase of the same work.

There are three parts, and each answers a plain question.

What is a platform, exactly? Not a repository, and not a program. A platform is a *composition*: you pick a governance surface (the rules), some workloads that prove the rules are honored, and optionally some business domains, and you seal them together into a single immutable artifact. Change the selection and you have a different platform, built from the same parts. The paper then asks what would let you say such a thing is *functioning*, and answers with conditions you can check rather than adjectives you can assert: it must be finite in what it admits, reachable from outside, able to answer questions about itself, and able to produce evidence of what it did.

What is a change, exactly? When such a system changes, the version already running is not merely the backdrop to the change — it is a *participant* in it. It supplies structure to reuse, it bounds what is admissible, and it is the authority against which any factual claim about the previous version is checked. The paper treats change as a function from one sealed baseline to the next, and draws one distinction that makes governed change auditable rather than just careful: the *function* is partial — not every problem produces a next version, and some are correctly refused — while the *process* is total. Every failure stops at a named stage with a stated finding. Nothing ever ends in an undefined state.

What did building it teach that reviewing it could not? This is the part that is hard to get from architecture. Building the platform surfaced capabilities that could not be reached at all the first time anyone tried to use them, a kind of change the governed lifecycle had no way to express, and rules a domain declared that nothing actually enforced. Every one of them sat on a path no one had ever executed. That is the paper’s most transferable finding: defects like these are a matter of *coverage*, not of maturity — they are not waiting to be outgrown, they are waiting to be walked into. Architecture establishes what must be true; realization establishes whether a particular build made it true. Treating those as one activity is a plausible reason so many systems are built from correct architectures and are nonetheless wrong.

Throughout, a reference implementation is cited as evidence and never as the subject. Each section separates three things and labels them: what the architecture *obliges* anyone to do, how this particular build *discharges* that obligation, and what is merely *incidental* to this build. Delete the second and third and a specification should remain. Appendix B is that deletion, performed.

Preface: how to read this paper

This is the third paper in a series and the first that is not about architecture.

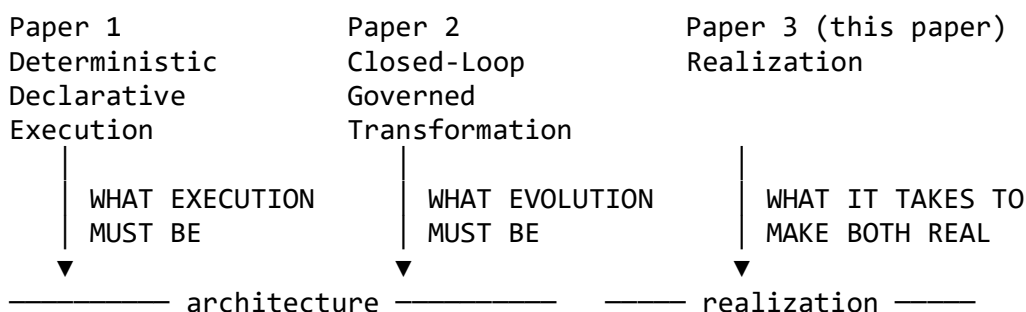


Figure 1 — Lineage. The companions establish the architecture; this paper establishes what it takes to realize it.

The companions [Ganti, 2026k; Ganti, 2026l] are complete accounts of their subjects, and both are published; they are cited above. This paper does not restate them. Where the argument requires an architectural result, it is cited and used. Readers who want to know *why* execution must be declarative, or *why* evolution must be transformation rather than authoring, should read the companions; this paper assumes both and asks what must be built.

Because this paper’s subject is realization rather than architecture, one further orientation is owed. The platform, the transformation toolchain, the snapshots, and the traces argued from here are open source at <https://github.com/protocol-governed-computing>. Nothing in Part III is reported on trust: a reader may rebuild a composition, exercise it across the boundary, and read the same evidence the findings were drawn from. A realization paper whose realization cannot be inspected would be asking for exactly the credence it claims to replace.

Two earlier papers [Ganti, 2026a; Ganti, 2026b] developed the same substrate under the name Protocol-Governed Systems. They remain a correct account of construction and execution and are frozen as the historical record. Nothing here retracts them. What has changed is that a realization layer now sits above the model they formalize, and describing that layer as a revision of them would misdescribe both. The term used throughout this paper is **Protocol-Governed Computing (PGC)**, which names the union of governed transformation above the protocol and declarative execution below it. Both companions carry it, on the grounds stated in [Ganti, 2026l, §17]: neither half is PGC on its own.

The three-move device

Every realization section separates three things and says which is which.

EVERY REALIZATION SECTION

OBLIGATION	REALIZATION	INCIDENTAL
what the architecture requires, naming no implementation	how this implementation discharges it	what binds no one and is an artifact of this build

survives the implementation

discarded when the implementation is replaced

Figure 2 — *The three-move device. What survives implementation obsolescence, and what does not.*

The ageing test. Delete every *Realization* and *Incidental* passage, and what remains must be a usable specification of what it takes to build a protocol-governed platform. Appendix B extracts exactly that. A paper about a reference implementation that cannot survive its reference implementation has failed at its own subject.

This device is offered as a methodological contribution and not merely as a drafting convention. Realization papers decay into manuals because they do not distinguish the obligation from the answer. Stating the distinction structurally, and providing the extraction as an appendix, is what makes the decay detectable.

1. Introduction

1.1 The question this paper asks

An architecture is a claim about what must be true. A realization is a claim that something *is* true of a particular construction. The two are related but not identical, and the gap between them is not merely engineering labor. It is where a correct architecture meets the facts that only construction can produce.

The companions leave that gap open by design. [Ganti, 2026k] establishes that a governed system evolves by transforming an executable baseline into its successor rather than by authoring a specification. [Ganti, 2026l] establishes that execution is traversal of a compiled protocol by an interpreter that decides nothing. Both are architectural: they say what must hold. Neither says what must be constructed, what properties distinguish a realized platform from a described one, or how one would know the construction had succeeded.

This paper asks:

What does it take to realize Protocol-Governed Computing as an actual functioning computational platform, and how would one know it had been achieved?

1.2 The thesis

Protocol-Governed Computing is realized not merely by implementing a governed execution platform, but by making **both the platform and the transformation of that platform** into governed, inspectable, evidence-producing computational objects.

Formally, PGC is the pair:

$$\boxed{\text{PGC} = \{B_n\}_{n \geq 0} + \mathcal{T}}$$

PGC comprises a *sequence* of governed platform baselines together with the governed transformation that relates each baseline to its successor:

$$\mathcal{T}: (B_n, P) \rightarrow B_{n+1}$$

The platform is the noun; transformation is the verb. Neither a single baseline nor the function alone is the object of study: a lone baseline is a governed program, and a lone function is a methodology. What this paper treats is the pair — a sequence of sealed states and the governed relation between consecutive members of it.

The asymmetry between noun and verb is deliberate and is the architecture rather than a defect in the framing. A platform is a state; a transformation is a change of state. A realization that supplies only the first is a governed program. A realization that supplies

only the second is a methodology. PGC requires both, and requires each to be governed by the same discipline.

1.3 Contributions

1. The Profiled Normative Platform (PNP) — a definition of what a realized platform *is*: a composition under a conformance profile rather than a repository, together with five checkable conditions under which it may be called functioning.

2. The baseline as a party to its own transformation — the proposition that B_n is not context for a change but a participant in it, supplying reuse, bounding admissibility, and serving as the reference for empirical claims about the existing system, with normative admissibility remaining with governance. Two consequences follow: that $\mathcal{T}$ is defined only relative to a *pinned* baseline, and that an existing implementation is part of the specification substrate for its own successor.

3. Determinacy conditions for governed transformation — what must hold at each stage for $\mathcal{T}$ to be defined, including a measure that answers whether a design determines the artifacts it schedules rather than whether it appears complete.

4. Partial function, total process — the distinction that makes governed evolution auditable: $\mathcal{T}$ is mathematically partial, while the transformation *process* is total with respect to outcome reporting.

5. Realization as a distinct epistemic activity — evidence, from constructing the platform, that architecture and realization establish different things, and that defect discovery in a governed system is a coverage property rather than a maturity one.

Part I — The Normative Platform

The noun. What a realized PGC platform is, and the conditions under which it may be called functioning.

2. From architecture to obligation

Obligation. The companions impose requirements that read as properties: execution decides nothing; the baseline is authoritative; behavior originates only in the protocol. A realization must convert each into an object that exists and can be pointed at.

The distance is larger than it appears. “The runtime decides nothing” is a property. A runtime that decides nothing is an artifact with a specific shape: it must have no branch that consults domain meaning, no table it populates by discovery, no fallback when a declaration is absent. Each of those is a construction decision that the property does not determine, and each can be got wrong while the property is still recited.

The obligations this paper takes from the companions are:

1. There exists a single authoritative compiled artifact that defines the system completely, and it is the only thing execution reads.
2. Behavior admissible at execution is exactly the behavior present in that artifact; nothing enters at execution time.
3. Authority is enumerated; no execution context confers it.
4. Execution produces evidence sufficient to check the claim that it conformed.
5. The system evolves only by transforming that artifact into a successor, under governance.

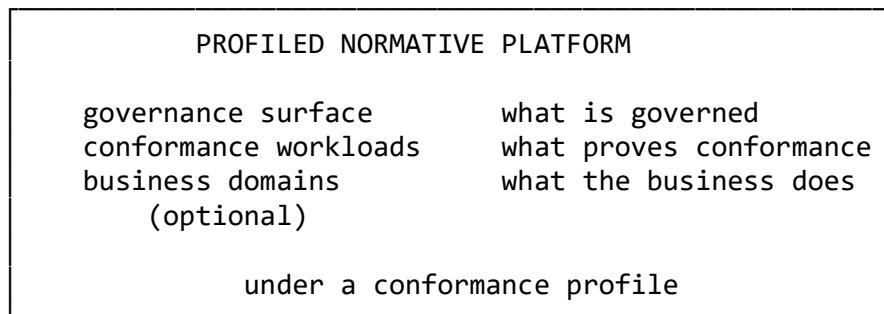
Parts I and II take these in turn. This section only fixes them as obligations so that later sections have something to discharge.

3. A platform is a composition, not a repository

Obligation. A realization must define the boundary of the thing being governed. If that boundary is drawn at a source repository, governance becomes a property of how code is stored, which is an accident of tooling.

The definition. A *Profiled Normative Platform* is:

$$PNP = \text{Governance Surface} + \text{Conformance Workloads} \\ + \text{Business Domains (optional)} \text{ under a Conformance Profile}$$



repositories, packages, directories
are where the material is kept –
outside the boundary, and irrelevant to it

Figure 3 — The platform boundary. Composition under a profile; storage is not part of the definition.

Three consequences follow, and they are the reason the definition is worth stating.

There is no single “the platform.” There are as many platforms as there are profiles. A profile selects a governance surface and the workloads that prove conformance to it; a different profile over the same material is a different platform, with a different admissibility set and a different snapshot identity. Asking “is the platform conformant?” is therefore malformed until a profile is named.

Repository structure is an implementation detail. The material composing a platform may be held in one repository or twelve; the composition is unchanged. This is not a stylistic preference. When the two boundaries are conflated, governance questions acquire spurious answers: a change is judged “internal” because it did not cross a repository boundary, although it crossed a governance one, or judged “external” for the opposite reason.

The composition is the unit of release. Because admissibility is a property of the whole, the parts cannot be released independently and remain meaningful. A governance surface released without the workloads that prove conformance to it has no evidence attached; a workload released against an unnamed surface conforms to nothing in particular.

Realization. The reference implementation composes a governance surface, a set of conformance workloads, and several business domains, each held in its own repository, assembled under a profile into a single snapshot. The repositories are not the platform and are never referred to as such; the build tooling takes composition roots as parameters rather than assuming a layout.

Incidental. The number of repositories, their names, and the choice to keep the governance surface separate from workloads are conveniences of this build. A single-repository realization satisfying the same obligations would be equally conformant.

4. The sealed snapshot as the boundary object

Obligation. There must exist one well-defined **baseline state**, and one sealed artifact that is its **authoritative executable representation** — not a description of that state, not a build output alongside others, but the object whose identity is the system’s identity at that moment.

The two are distinct and the distinction is load-bearing:

$$B_n \xrightarrow{c} S_n$$

B_n is the complete *authored protocol state* — the artifacts a transformation produced, in the form a human or a generator wrote them. S_n is its *sealed, content-identified compiled representation*. Only S_n is executed; only S_n is what a pin names; only S_n has an identity derived from content. A reader who conflates them will ask whether “the baseline” means the source protocol state or the compiled artifact, and the answer is that both exist, with different roles: the baseline is what a transformation produces and what the next transformation reads for reuse; the snapshot is what execution and pinning address.

This sealed artifact is the hinge of the entire model, and its role is easiest to see when both halves of the paper are drawn together.

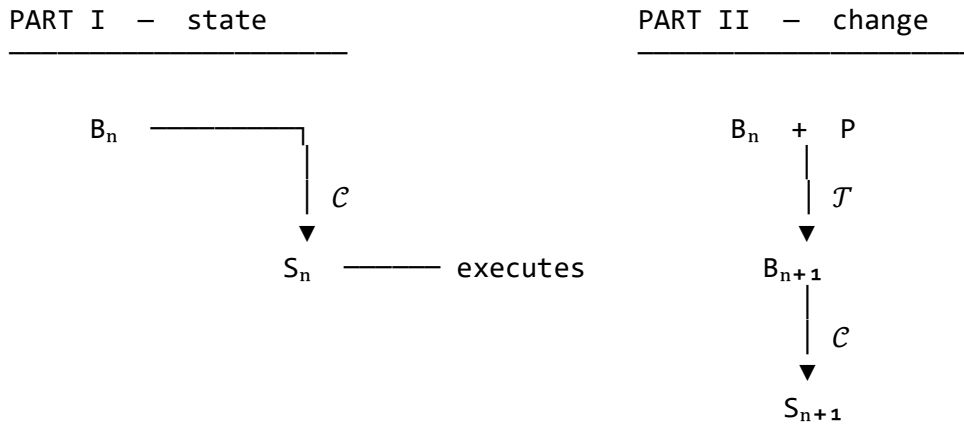


Figure 4 — The snapshot as the boundary between state and change. It is the sealed representation of a baseline, which is what makes pinning and comparison expressible.

Calling the snapshot “the sealed representation of a baseline” rather than “the compiled output” changes what can be said. To pin a baseline is then to name a snapshot; to compare two baselines is to compare two sealed objects rather than two intentions; and the recursion of §11 becomes a sequence of named things rather than a narrative about releases.

Obligation, stated operationally. For the seal to mean anything:

- the snapshot must have an identity derived from its content, so that two claims about “the same baseline” can be checked rather than trusted;

- assembly must be reproducible from committed source together with declared, pinned build inputs, so the identity is a property of the material rather than of the machine that built it;
- nothing may be admitted at execution time that was not present at seal time, and this must be a structural property rather than a discipline.

Realization. Assembly produces a content-derived snapshot identifier and verifies the composition round-trip on every build. Reproducibility is checked by deleting every build output and rebuilding from committed source together with the declared, pinned build inputs, expecting an unchanged identifier — a practice consistent with the goals of the reproducible-builds literature [Lamb & Zacchiroli, 2022]. The execution boundary reads the sealed snapshot once, at startup, and reports the identity it booted with, so that a stale process is a one-line comparison rather than an inference.

Incidental. The particular hash construction, the on-disk layout of the snapshot, and the decision to report the identity in a startup banner.

A realization hazard worth recording. A platform artifact may be compiled into more than one place within a single composition — once per domain that imports the governance surface, for instance. Nothing in the architecture forbids this, and nothing about it is visible until the copies disagree. In the reference implementation they did: editing one governance artifact and recompiling a single domain left five copies of it in two versions, and the composition assembled, verified round-trip, and reported conformance while the published capability surface answered from a stale copy. The obligation this yields is general: **every copy of an identity within a composition must be identical, and the assembler must check it rather than the builder remember it.** §19 returns to why architectural review does not surface this class of defect.

5. Closure: what a platform must be finite in

Obligation. Governance is tractable only over a finite surface. A realization must therefore close something, and must say what.

Three closures are load-bearing:

- **The capability surface.** The set of side-effecting capabilities is fixed for a given profile. A domain may compose them; it may not introduce a new one.
- **The concern vocabulary.** The categories of execution concern are fixed. New behavior arises by composition within the vocabulary, never by adding a category at runtime.
- **The admissibility set.** The set of admissible execution paths is exactly what compilation constructed; execution adds none.

Closure is what converts governance from an aspiration into a finite question. Where the surface is open, “what can this system do?” has no answer short of reading all of the code — which is precisely the position the architecture exists to escape [Ganti, 2026]. This is the same move that capability-based security makes against ambient authority [Dennis & Van Horn, 1966; Miller, 2006], generalized from authority to behavior.

The cost, stated plainly. Closure has a price, and a realization paper should name it: *the capability that does not yet exist is invariably the one a new domain needs*. In the reference implementation this occurred twice. A domain needed the current time, which no capability provided and which cannot be a pure transform; and a domain needed to change part of a stored record without replacing it, where the store offered whole-value replacement at a key and field-merge across a filter, but nothing that merged fields at a key.

Both cases resolved the same way, and the pattern is the obligation: **when a domain needs a neutral mechanism the substrate lacks, the substrate gains it — the domain does not improvise**. The alternative was available in the second case and was rejected: the filtered merge could have been used, and would have been correct only because that domain happened to guarantee that its filter matched exactly one record. That correctness would then have travelled with the pattern to every domain that copied it, without the guarantee. A business domain compensating for a missing substrate operation with its own invariant produces exactly this: a correction that carries an unstated promise.

Realization. Six side-effecting capabilities, closed by an invariant that enumerates them. Both gaps above were closed by extending the neutral surface, not the domain — in the second case by adding one operation to an existing capability, which left the enumerated set unchanged and therefore required no governance amendment.

Incidental. Which six, and the specific operation vocabulary of each.

6. Reachability: a platform that cannot be reached is not functioning

Obligation. A platform whose acts cannot be invoked from outside it is a library. If governance is to extend to *what may be asked of the system*, the boundary must be a governed contract rather than a wrapper around one.

The requirements are:

- **A public act has a name of its own.** What a caller names must be distinct from what performs the act, so that the performer may be replaced without any caller changing.
- **What a caller may send is declared, not inferred.**
- **What a caller is told is a governed classification**, drawn from a closed set, carrying no transport semantics.
- **The adapter is non-authorial.** Protocol mechanics only; it decides nothing about meaning.

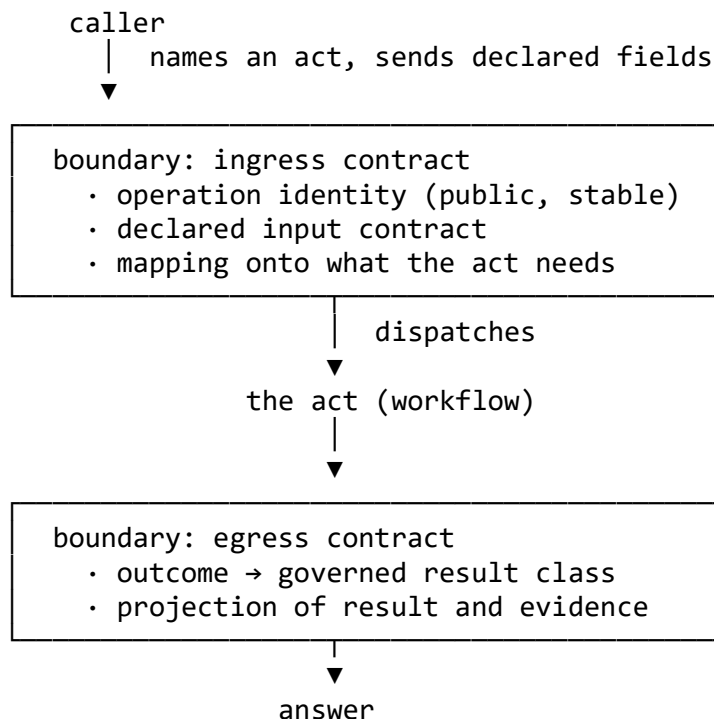


Figure 5 — The governed boundary. The public name and the act are separate declarations, which is what allows one to change without the other.

Why the separation is load-bearing. If a caller names the performer, then re-pointing the act breaks every caller, and the system’s public surface becomes a function of its internal structure. Separating them means a public name can outlive several implementations of what it reaches. In the reference implementation this was exercised deliberately: two public names were bound to a single act, because the business regarded the two as

distinct in kind while the machinery performed them identically. Callers cannot tell, and need not.

What a client is permitted to be. A client collects, sends, and renders. It holds no rule. This is stronger than it sounds and is routinely violated for good reasons: a form that validates before sending is a second opinion the governance never approved, and the two opinions will eventually disagree. The obligation is that **the platform judges and says which field was at fault; the client displays that judgment.** A client that validates has become a second, ungoverned author of admissibility.

Realization. Boundary contracts are compiled artifacts sealed into the snapshot and read once at startup. A route may carry a fixed act, or admit a family of acts whose identity travels in the request, with the namespace checked as an *admission constraint* rather than used as a dispatcher — the adapter never branches on what an identity means. The reference client carries no validation attributes at all, so that a missing required field is refused by the platform, which names it.

Incidental. The wire protocol, the route table format, and the choice to serve a browser client at all.

7. Inspection: a platform must answer questions about itself

Obligation. A governed composition that cannot be interrogated is governed only in principle. Realization requires that the questions governance asks — what exists, what depends on what, what consumes this store — have answers obtainable *through* the governance model rather than around it.

The failure this prevents is specific and common: a consumer that needs a fact about the composition reaches into the compiler’s internals to get it, and thereafter the compiler cannot change without breaking a consumer that was never supposed to know it existed.

Realization. Inspection is realized as a domain, not a utility: it declares operations with identities, reaches the composition through the same boundary machinery as any other domain, and is the only sanctioned path to a compiled fact. The transformation tooling consumes it and imports no compiler module; that constraint is the acceptance test for the inspection work.

Incidental. The operation names, and the decision to expose inspection over the same transport as business acts.

A realization hazard. An inspection surface can be present, well-formed, and wrong. In the reference implementation the operation answering “what consumes this store” returned an empty answer for fourteen of fifteen stores, because the join it performed recognized only one of the three ways a binding may declare where it writes. Every call succeeded. The obligation: **an inspection answer that is empty must be distinguishable from one that is unknown**, and a realization should test its inspection surface against facts established independently, because a confident empty answer is worse than an error.

8. Evidence: execution must produce proof

Obligation. A claim that execution conformed is worth what it can be checked against. Realization requires that every execution produce a record sufficient for an independent party to verify the claim, and that the record be a governed artifact rather than a log.

This is provenance in the sense the provenance literature intends [Moreau & Missier, 2013]: not a diagnostic byproduct, but a structured account of what produced what, retained because someone will need it.

Obligation, in detail. The record must identify the act, the path taken through it, and the effects produced; must be produced by the same traversal that produced the effects, rather than reconstructed; and must be addressable, so that an answer given to a caller can be tied to the record of the execution that produced it.

Realization. Every execution writes a trace addressed by identity; the boundary exposes a *reference* to that trace rather than its content, so that an answer stays small and the evidence stays retrievable. Where the platform serves a client, the reference resolves to the trace, and the record of what happened is one request away from the answer reporting it.

Incidental. The trace format, its storage layout, and the reference syntax.

9. What “functioning” means

Obligation. A realization paper must say what it means to have succeeded, in terms that can be checked rather than asserted.

A Profiled Normative Platform is **functioning** when it satisfies five conditions:

COMPOSES	the composition assembles under a named profile into a single artifact
SEALS	that artifact has a content-derived identity, reproducible from committed source and pinned inputs
EXECUTES	the declared conformance workloads execute and produce the effects their declarations require
ANSWERS	it can be interrogated about its own contents through governed operations
PROVES	every execution yields evidence sufficient to check that it conformed

Figure 6 — The five conditions. A platform failing any one is not functioning, whatever else is true of it.

The conditions are deliberately modest, and the third is narrower than it may first appear. EXECUTES requires that the *declared conformance workloads* run and produce what their declarations require — not that every capability the platform declares has been demonstrated. That weaker reading is the only one the evidence supports, and it harmonizes with §18: a governed platform is correct where exercised and unverified elsewhere. A platform can satisfy all five conditions and still contain capabilities no one has ever invoked.

The conditions do not require that the platform be useful, complete, or correct in its business logic. They require that it be *governed in fact* rather than in description. A platform that composes and seals but cannot be reached is a well-formed artifact and not a functioning platform; one that executes but produces no evidence has no way to substantiate the claim its architecture makes.

Part II — Governed Transformation

The verb. How a platform becomes its own successor, and what must hold for that to be defined.

10. The baseline is a party to its own transformation

Obligation. Governed evolution requires that the current baseline participate in producing the next one. This is stronger than requiring that it be consulted.

Conventional change treats the existing system as context: something to be read, understood, and then modified. Governed transformation treats it as a participant with three specific roles.

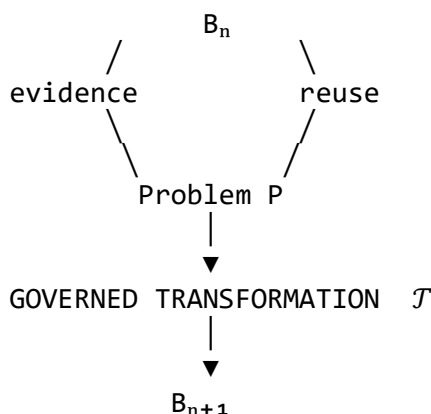


Figure 7 — The baseline as a party. It supplies reusable structure and the evidence against which every empirical claim about the predecessor is checked, and it bounds what the transformation may admit. What the successor may become remains governance’s to decide.

- **It supplies reuse.** What already exists and satisfies a need is not rebuilt; the transformation must find it and say so.
- **It bounds admissibility.** What the next baseline may contain is constrained by what this one declares — its closed capability surface, its vocabulary, its governance.
- **It is the reference for empirical verification.** Every belief the change asserts *about the existing system* is resolved against it, and resolved before the change is designed.

The third role requires a boundary that is easy to lose. The baseline settles what *is*; it does not settle what *may be*:

Baseline $\rightarrow$ empirical truth about the predecessor

Governance → normative admissibility of the successor

A change may verify every belief against the baseline and still be inadmissible, because admissibility is decided by the governing artifacts and the conformance profile, not by what happens to exist. Conflating the two would make the existing system the authority on what the next one may become, which is precisely the drift governed transformation exists to prevent.

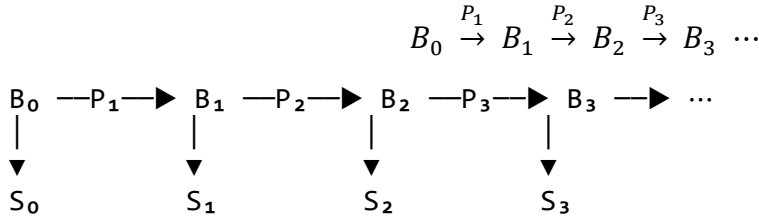
Two consequences follow, and both are formal rather than procedural.

$\mathcal{T}$ is defined only relative to a pinned baseline. A transformation validated against a baseline it was not pinned to is not a weaker transformation; it is undefined. The facts it verified were verified against something else. A realization must therefore make the pin explicit, carry it with the change, and refuse to proceed when the observed composition does not match.

A pin belongs to the change that was judged against it, and must not be advanced afterwards. This is subtle and was got wrong once in the reference implementation before being corrected. When a change completes, the composition moves — *because that change moved it*. Re-pinning the completed change to the new composition destroys the record of what it was actually judged against, and worse, asserts a re-reading of facts the change itself falsified: a belief recorded as “this function has no external boundary,” true when verified, is false immediately after the change that added one. The correct discipline is that the successor change pins the composition its predecessor produced. An *in-flight* change whose composition moves for unrelated reasons may legitimately re-pin, because nothing it asserted has been falsified by its own action.

The stronger claim. The existing implementation is part of the specification substrate for its own successor. This is the sense in which governed transformation differs from specification-driven development as conventionally practiced [Schmidt, 2006; France & Rumpe, 2007]: the specification is not authored against a blank page and reconciled with an implementation afterwards; it is derived against an executable predecessor that supplies half of it.

11. The recursion, and what survives it



each baseline seals to a snapshot; each arrow is an inspectable governed transformation

Figure 8 — The recursion. A system's history is a sequence of governed transformations, not a sequence of informal releases.

Preserved across composition. The closure properties of §5 — a transformation may compose the capability surface but not extend it, so the surface at B_{n+1} is the surface at B_n unless a governed substrate change intervened. Authority enumeration. The property that execution adds no paths.

Re-established at each step. Admissibility, which is recomputed by compilation over the whole composition rather than inherited; evidence, which attaches to executions and not to baselines; and every belief a change asserts, which must be re-verified against the pin rather than carried forward from a predecessor's verification.

The distinction matters because it says what an auditor may assume. Properties in the first class hold at B_n for all n once established. Properties in the second must be shown at each step, and a realization that carries them forward implicitly has weakened the guarantee without saying so.

12. The shape of a governed change

Obligation. The transformation must proceed in stages, each of which either reorganizes what is already stated or decides something new, and never both. The reason is auditability: a stage that reorganizes and decides in the same act leaves no way to tell which of its outputs were derived and which were invented.

The stages of the reference lifecycle run from a business problem to an authoring mandate. Their *rationale* is the subject of the transformation companion [Ganti, 2026k] and is not restated. What this paper contributes is the condition under which each transition is *defined*.

13. Determinacy: what must hold for $\mathcal{T}$ to be defined

$\mathcal{T}$ is partial. It is defined at a transition only when that transition's condition holds.

stage	determinacy condition for the transition
seed	reorganizes, never decides; no unanswered question carried forward as though settled
↓	
change request	projection from the seed is lossless and mechanical – nothing added, nothing dropped
↓	
domain model	every declared belief resolved against the pinned baseline: verified, absent, or explicitly insufficient
↓	
analysis	saturation – no unresolved critical gap, no open question, no dependency expansion in the last pass, every inference promoted or carried with a stated reason
↓	
business model	every committed capability consolidated; none silently dropped between stages
↓	
business intent	every in-scope capability has declared intent
↓	
governance intent	every in-scope capability placed; every claim of satisfaction names an existing artifact
↓	
design intent	the design uniquely determines the artifacts it schedules, and amends nothing it does not restate ← construction completeness
↓	
authoring mandate	every designed artifact scheduled; dependencies precede dependents
↓	
B_{n+1}	

Figure 9 — *Determinacy conditions. $\mathcal{T}$ is defined at a transition only if its condition holds; failure at any stage is reported at that stage.*

Several of these deserve comment because they are easy to state and hard to realize.

Reorganizes or decides, never both. The first stage restates a human-authored problem into governed registers. It must add no content. A question the problem does not answer becomes a recorded clarification request, never a filled-in guess; a belief about the existing system must not be promoted to a fact. Realizing this requires the machinery to *refuse* a document carrying an unanswered question, because a question answered by the process that asked it is indistinguishable from a fact.

Every belief resolved against the pin. The stage that verifies claims about the existing system is the spine of the whole lifecycle, and its output is three-valued: verified, absent, or insufficient evidence. The third value is what prevents a change from proceeding on an assumption that merely went unchallenged.

The design determines its artifacts. This is the condition at the construction transition and the one that admits a measure. A design may be complete in the sense that every register is populated and still fail to determine the artifacts it schedules — the two are different claims, and only the second matters. The measure used here is derived from the shape the generator emits, walked leaf by leaf, rather than from a hand-maintained checklist.

The measure, defined. Let D be a design together with its mandate, and let $A(D)$ be the set of artifacts that mandate schedules or that design amends. For each $a \in A(D)$, let the generator emit its machine block and let $L(a)$ be the set of *leaves* of that block — every terminal position reached by walking it, addressed by path. Let

$$L(D) = \bigsqcup_{a \in A(D)} L(a)$$

A leaf is **determined** when it carries a value, or when the design declared it empty:

$$\text{determined}(\ell) \Leftrightarrow \text{value}(\ell) \neq \emptyset \vee \ell \in \text{declaredEmpty}(D)$$

Construction completeness is the proportion of determined leaves:

$$CC(D) = \frac{|\{\ell \in L(D) : \text{determined}(\ell)\}|}{|L(D)|}$$

$$\boxed{CC(D) = 1 \Leftrightarrow D \text{ is construction-complete}}$$

and the transformation **refuses below 1**.

Three properties of this definition carry the weight, and each corrects a way the measure can be made to lie.

$L(D)$ is derived, never declared. The leaf set *is* the shape the generator emits, walked at measurement time. A hand-maintained requirement list is a second description of the generator that drifts from it the moment either changes — and in the reference implementation it did, reporting full determinacy while the generator could in fact reproduce only a small fraction of the artifacts it was asked for. Deriving $L(D)$ from the emission makes drift impossible by construction: the measure cannot disagree with the generator because it is computed from it.

Declared emptiness is distinguished from omission. An empty policy and an unstated policy are identical in the output and different in meaning, so the difference must be a *declaration*. A builder that knows a field to be intrinsically empty — reserved by the protocol version, say — records it in `declaredEmpty( $D$ )`; a design that simply failed to state a value does not appear there. Without this distinction the measure either penalises correct designs or excuses incomplete ones.

An artifact with no builder contributes an undetermined element. If construction has no builder for an artifact’s family, that artifact contributes one element to $L(D)$ and it is never determined. Omitting such artifacts from the measure entirely — which is what silently skipping them amounts to — lets a design score $CC(D) = 1$ while naming an artifact that cannot be produced at all. An unbuildable artifact is the strongest possible construction failure, and it must not read as no failure.

The refusal at $CC(D) < 1$ is the mechanism by which a generator is prevented from becoming a second, ungoverned author: a generator permitted to fill a gap has authored a design decision that no governance approved.

Amendment must not narrow. A change that re-renders an existing artifact renders it *whole*. A design stating only the delta therefore deletes everything it did not restate — and does so at full determinacy, because determinacy never looks at what already exists. The realization must compare the render against the composition and report facts that exist now and the design does not state. In the reference implementation this check caught a design that would have silently dropped fifty-one facts from a build manifest.

14. Amendment is not authorship

The finding. A governed lifecycle can be structurally incomplete while every stage appears fully specified.

If the machinery assumes

Change $\Rightarrow$ NewArtifact

then it cannot express

Change $\Rightarrow$ ExistingArtifactModified

and the incompleteness is invisible until a change of the second kind is attempted. In the reference implementation, three consecutive stages each demanded at least one row in the register naming what the change creates. A defect correction creates nothing. The change was inexpressible — not difficult, not inelegant: **inexpressible** — and the only alternatives were to manufacture an artifact to satisfy the rule, or to version-bump the corrected artifact and cascade the change through everything that named it.

What made this diagnosable was that the *generator* already knew better: it rendered what a mandate schedules **and what a design amends**, with an explicit note that an amended artifact is never a build step. The construction layer had encountered the case and handled it; the governance layer forbade it.

The obligation. A governed lifecycle must admit the change that creates nothing. Concretely: the registers naming what a change *creates* must be permitted to be empty, while the biconditionals that bind them to later stages remain — nothing may be introduced that was never named, and nothing scheduled that was never designed. What is removed is only the demand that the sets be non-empty.

Why this belongs in a realization paper. No architectural review would have found it. Every stage was specified, every rule was justified, and the composition of the rules excluded a legitimate class of change. That composition is only visible when something in the excluded class is attempted.

15. Partial function, total process

$$\mathcal{T}: (B_n, P) \rightarrow B_{n+1} \quad \text{partial}$$

$$\text{TransformationProcess}(B_n, P) \rightarrow \text{Outcome} \quad \text{total}$$

Not every problem yields a next baseline, and a model claiming otherwise would be false. But the *process* is total with respect to outcome reporting:

TransformationProcess(B_n , P) yields exactly one of:

B_{n+1}	success
inadmissible at stage k	with stated findings
unresolved obligation at k	named
failure at stage k	located

never: ???

Figure 10 — Total process. A transformation that cannot proceed says where and why.

This is the distinction that makes governed evolution **auditable rather than merely careful**. A careful process reduces the chance of an undiagnosed failure. A total process eliminates the category: every outcome is a stated one, at a named stage, in a form another party can check.

The realization requirement is that failure be *located and reported*, not merely detected. A stage that refuses must say which rule refused, over which row, and why — and must report every finding in one pass rather than the first, because a design is owed all of what it fails rather than an iterative reveal.

16. The three functions, composed

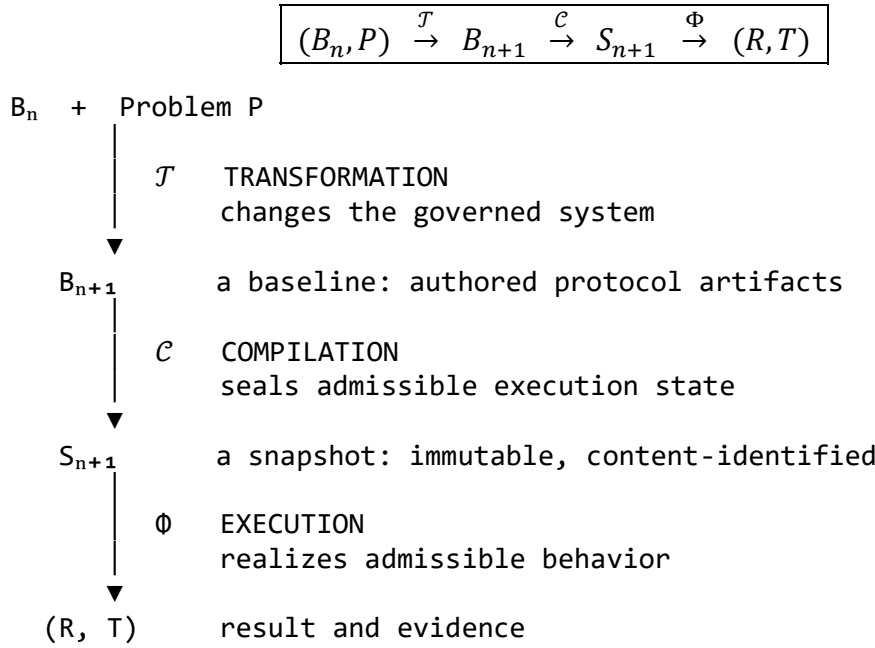


Figure 11 — The three functions. Each governed, each inspectable, composing into the life of a governed system.

	function	changes	partiality
$\mathcal{T}$	$(B_n, P) \rightarrow B_{n+1}$	the system	partial — not every problem yields a baseline
$\mathcal{C}$	$B_{n+1} \rightarrow S_{n+1}$	nothing; seals	partial — inadmissible artifacts have no compiled form
Φ	$(S_{n+1}, I, \sigma) \rightarrow (R, T)$	governed state	total over admitted inputs

What the composition guarantees that no single function does. Each function alone yields a familiar and insufficient thing. $\mathcal{C}$ and Φ together give a governed program: behavior constructed rather than authored, executed by an interpreter that decides nothing. That is the subject of [Ganti, 2026] and of the earlier PGS work. $\mathcal{T}$ alone gives a methodology for producing specifications. Composed, they give something neither provides: **a system whose every state is sealed and executable, and whose every transition between states is itself a governed, inspectable object.** The history of the system becomes a chain of such objects rather than a narrative about releases.

On the prior formal model. Earlier work formalized this substrate as a tuple of functional responsibilities covering authoring, governance, compilation, execution, and structure

[Ganti, 2026b]. That decomposition is correct and is properly contained here: it describes $\mathcal{C}$ and Φ and the material they operate on. What it does not describe is $\mathcal{T}$, because at the time it was written the transformation of one baseline into the next had not been realized and could not be formalized from first principles. Nothing in it is retracted. It was complete as an account of construction and execution, and incomplete as an account of a system's life.

Part III — What Realization Surfaced

17. Defects surfaced only through realization

A realization paper reporting only successes is a brochure. This section reports what constructing the platform exposed, because the classes are instructive even where the specifics are not.

Five principal defects are discussed below; three further realization findings are recorded in Appendix C, which is the complete register. Each of the five lived on a path that had never been executed.

Determinacy unsatisfiable for an entire artifact family. The construction measure required, for each artifact, that every leaf be determined or declared empty. The builder for one artifact family emitted a reserved field that was always empty and had no way to declare it so. Every artifact of that family therefore measured one short of total, against a threshold that refuses below total. The consequence was not a warning: **no artifact of that family could ever have been constructed by anyone**, and the fact was invisible because no one had constructed one.

Structured payloads silently malformed. The same builder mapped fields by name into a flat structure. A field addressed by a dotted path — naming a value inside an object — became a key with a dot in its name, and the object the act expected was simply absent. Correct for a payload one level deep; wrong for any other; and the only prior use had been one level deep.

Constants delivered as prose. A design states the values an act needs that a caller must not supply. The builder copied those cells verbatim, so a value stated as “*constant ACCEPTED*” reached the act as the string “constant ACCEPTED”. The design language had no convention distinguishing a value from a description of a value, and nothing had ever needed one.

Declared events that could not fire. Domains declared events for the moments they recognized. The generator wrote, on each terminal node, a field that no constitution declared, no compiler assertion checked, and no runtime read — and never wrote the one field the platform actually consults. Every domain the generator produced therefore declared events it could never fire, while every hand-authored domain fired them correctly. The field that worked was the one nothing wrote. Nine declared events across two domains were unreachable, and every document check passed.

Rules declared and never enforced. Two rules in a single subdomain were stated in the governing documents and realized in no artifact: that a rejection must state its grounds, and that a person keeps the details they were admitted with. Every phase rule passed, because a rule that no step consults is a true statement *about* an artifact rather than a property *of* one. The second was found only by executing the function and reading what it

left behind: recording a decision replaced the person's whole record, silently destroying the name and preferences they had registered with, and succeeding every time.

18. Coverage, not maturity

$$\text{DefectDiscoveryRate} \approx f(\text{UnexercisedSurface})$$

The five principal defects share a property more informative than any of them individually: **each lived on a path that had never been executed**. The three further findings in Appendix C share it too. None was a corner case, a race, or a subtle interaction. Each was an entire capability non-functional the first time anyone used it.

That yields a general statement about governed systems, and it is the more useful contribution of this part:

A governed implementation is **correct where exercised and unverified elsewhere**. The rate at which realization defects are discovered is a function of unexercised surface, not of the implementation's maturity.

The distinction matters because the two have opposite remedies. If the discovery rate reflected immaturity, the remedy would be time and polish. Because it reflects coverage, the remedy is to exercise surface deliberately — and the surface to exercise is enumerable, because a governed system declares its own registers, families, and stages.

What would signal approaching completeness is therefore not the absence of defects but a particular event: *a change that exercises a genuinely new shape and finds nothing*. Until that occurs, the correct expectation is that the next new shape will find something.

The class of instrument that raises coverage. Where a system holds a corpus of *inadmissible* cases — documents that must be refused, each isolating one rule — it may hold no *maximal admissible* case: a single synthetic instance populating every register and naming every artifact family, whose only job is to traverse the whole generator once. Four of the five defects above would have been caught by one such case, at build time, in a single run, rather than one per change over several days. This is offered as an example of the class rather than as a prescription: the general principle is that coverage instruments should be shaped by the enumerable surface the system declares about itself.

19. Realization as a distinct epistemic activity

The five principal defects, and the coverage principle they yield, support the paper's methodological claim:

Architecture establishes invariants. Realization establishes whether an implementation instantiates them. These are different activities, and treating them as one is a plausible explanation for why architectures are so often correct while the systems built from them are not.

Each defect in §17 occurred *beneath* a correct architectural statement. The architecture said declared events are announced at the moments a domain recognizes; the generator wrote a field nothing read. The architecture said a design determines its artifacts; the measure could not be satisfied for one family. The architecture said a governed change proceeds through admissible stages; the stages composed to exclude a legitimate class of change. In each case the invariant was right and uninstantiated.

This is not an argument that architecture is unimportant. It is an argument that an architecture *plus a claim of conformance* is two claims, and that the second requires its own evidence. Formal verification establishes properties of an implementation against a specification [Lamport, 2002; Jackson, 2006]; realization evidence, as used here, is weaker and cheaper: it establishes that the paths a system declares have been traversed at least once, and reports what that traversal found. For systems that declare their own structure, this is unusually tractable, because the surface to be covered is enumerable rather than inferred.

There is a long-standing observation that a rational design process is something one fakes in the writing rather than follows in the doing [Parnas & Clements, 1986]. Governed transformation makes a narrower and more checkable claim: the *record* of the process is produced by the process, so what is written down is what happened, including the refusals.

20. What the reference implementation demonstrates, and what it does not

Demonstrated. A composition of a governance surface, conformance workloads, and business domains assembling under a profile into a content-identified snapshot, reproducible from committed source against declared, pinned build inputs. A platform reachable across a governed boundary, answering inspection queries about itself, and producing an evidence trace addressable from the answer it returns. A sequence of governed transformations of a single business subdomain, each pinned to its predecessor's composition, including one that authored no artifact at all and changed two lines of one. A substrate extension performed on the neutral surface rather than in the domain that needed it. Failure semantics exercised throughout: refusals located at named stages, with findings reported in one pass.

Not demonstrated. Composition across many business subdomains at scale; more than one conformance profile over the same material; concurrent transformation by multiple authors; adversarial use of any kind. The reference implementation has not been operated by anyone other than its author, and the claims in this paper are correspondingly bounded: they are claims about what a realization requires and what one realization exhibited, not claims about production operation.

A note on the deferred. Three declared rules in the demonstrated subdomain remain unenforced within it, and are recorded rather than concealed. Two are enforced at the boundary and not in the act; one is unenforced entirely. They are carried deliberately to the next transformation, where the first real consumer of that subdomain's state will make them testable against data rather than against a testbed. A realization paper that reported the subdomain as complete would be describing a different system.

21. Conclusion

The architecture of protocol-governed computing was established in two companion papers: execution as declarative traversal of a compiled protocol, and evolution as the governed transformation of an executable baseline into its successor. This paper asked what it takes to realize both, and what one learns by doing so.

The answer has two halves that cannot be separated. A **Profiled Normative Platform** is a composition under a profile, sealed into an identified snapshot, reachable across a governed boundary, able to answer questions about itself, and producing evidence of what it did — five conditions, each checkable, none satisfied by description alone. **Governed transformation** is the same platform becoming its successor, with the current baseline participating rather than observing: supplying reuse, bounding admissibility, and standing as the reference for empirical claims about what exists, while governance retains authority over what may be admitted. The transformation function is partial while the transformation process is total, which is what makes governed evolution auditable rather than merely careful.

Composed, the three functions — transformation, compilation, execution — give an account of a governed system's life in which every state is sealed and executable and every transition between states is itself a governed object. That is a stronger claim than either companion makes alone, and it is available only once both have been realized.

The third contribution is methodological and was not anticipated. Constructing the platform surfaced defects that architectural review had not surfaced, all of the same class: correct invariants, uninstantiated. That regularity yields a statement worth carrying beyond this system — a governed implementation is correct where exercised and unverified elsewhere, and the rate at which its realization defects surface is a function of unexercised surface rather than of maturity. It follows that realization deserves treatment as an activity in its own right, with its own evidence, rather than as the labor that follows design.

Architecture states what must be true. Realization is where one finds out.

Appendix A — The reference implementation

Purpose. The implementation is cited as evidence for the obligations in Parts I and II and as the source of the findings in Part III. It is not the subject of this paper, and no claim here depends on any of its particular choices. It is open source at <https://github.com/protocol-governed-computing>, so the compositions, snapshots, traces, and findings this paper argues from are inspectable rather than reported on trust; each repository carries an architecture document written for a reader approaching it without prior familiarity.

Composition. A governance surface declaring the closed capability set, the concern vocabulary, and the constitutional invariants. Conformance workloads whose execution proves conformance to that surface. Business domains realizing business functions. A protocol compiler producing domain projections; an assembler composing them into a snapshot; a runtime executing the sealed snapshot; a transport boundary; an inspection domain; and a transformation toolchain implementing the staged lifecycle of §12–13.

Reproducing a baseline. Compile the governance surface, compile each domain that declares source, assemble. Deleting every build output and rebuilding from committed source, against the declared pinned build inputs, is expected to yield an unchanged snapshot identity; this is the reproducibility check referenced in §4. Reproducibility across differing tool versions has not been demonstrated and is not claimed.

Exercising a platform. Serve the composition across the transport boundary, invoke a declared act, and follow the evidence reference in the answer to the trace of the execution that produced it. The five conditions of §9 are checkable in that order.

Scope of evidence. Single author, single operator, one conformance profile, one business domain transformed repeatedly. See §20 for what this does and does not support.

Appendix B — Realization requirements, extracted

The ageing test made mechanical: every obligation from the body, with no implementation.

Platform

1. The platform boundary is a composition under a named conformance profile, not a storage boundary.
2. There exists one artifact that is the baseline, with an identity derived from its content.
3. Assembly is reproducible from committed source together with declared, pinned build inputs.
4. Nothing is admitted at execution time that was not present at seal time, structurally.
5. Every copy of an artifact identity within a composition is identical, and the composition checks this rather than relying on build discipline.
6. The capability surface, the concern vocabulary, and the admissibility set are closed for a profile.
7. When a domain requires a neutral mechanism the substrate lacks, the substrate is extended; the domain does not compensate with a local invariant.
8. A public act has an identity distinct from whatever performs it.
9. What a caller may send is declared; what a caller is told is a governed classification from a closed set.
10. The boundary adapter is non-authorial: protocol mechanics only.
11. A client holds no rule, performs no validation, and stores nothing the platform holds.
12. The composition is interrogable through governed operations, and no consumer reaches into a compiler's internals for a fact.
13. An empty inspection answer is distinguishable from an unknown one.
14. Every execution produces evidence sufficient to check that it conformed, produced by the same traversal that produced the effects, and addressable from the answer returned.
15. A platform is functioning when it composes, seals, executes, answers, and proves.

Transformation

16. The current baseline participates in its own transformation: supplying reuse, bounding admissibility, and serving as the reference for verification.
17. A transformation is defined only relative to a pinned baseline, and refuses to proceed when the observed composition does not match the pin.
18. A pin belongs to the change judged against it and is never advanced after that change completes.
19. Each stage either reorganizes what is stated or decides something new, never both.

20. A question the problem does not answer is recorded as an open question, never filled in.
21. Every belief about the existing system is resolved against the pin, three-valued: verified, absent, or insufficient evidence.
22. A design determines the artifacts it schedules; determinacy is derived from what the generator emits rather than from a maintained checklist.
23. A leaf declared empty is distinguishable from a leaf left undetermined.
24. A generator may not fill a gap the design left; refusal below totality is the mechanism.
25. An amendment renders the artifact whole, and the realization reports facts that exist and the design does not restate.
26. The lifecycle admits a change that creates nothing, while retaining the constraints binding what is created to what was named and scheduled.
27. The transformation function is partial; the transformation process is total with respect to outcome reporting, locating every failure at a named stage and reporting all findings in one pass.
28. Properties preserved across the recursion are distinguished from those re-established at each step, and the latter are re-established rather than inherited.

Appendix C — Findings register

#	class	what was wrong	rule now in place
1	construction determinacy	a reserved always-empty field could not be declared empty, so no artifact of that family could reach total determinacy	the builder declares the emptiness it knows to be intrinsic
2	structured payload	a dotted field name became a flat key; the expected object was absent	dotted paths are expanded into the structure the act reads
3	value versus description	design cells were copied verbatim, delivering prose where a value was meant	a cell is read as a value unless it is a substitution token
4	dead field / silent absence	a field nothing read was written on every terminal node; the field the platform reads was never written	the generator writes the read field; the dead one is removed from the model and from every artifact carrying it
5	lifecycle expressiveness	three consecutive stages demanded at least one created artifact, excluding amendment	those registers may be empty; the binding constraints between stages are retained
6	declared, unenforced	rules stated in governing documents and realized in no artifact	found by execution validation; recorded, and closed by amendment where a consumer exists
7	composition integrity	copies of one artifact identity disagreed while the build passed every gate	the assembler compares copies and refuses a composition whose copies disagree

#	class	what was wrong	rule now in place
8	inspection blindness	an inspection operation answered for one store in fifteen, confidently and emptily	the join resolves every declared form, and precision is applied only where a binding is ambiguous

Appendix D — Vocabulary

Only terms this paper introduces or sharpens. The companions' vocabularies are referenced, not restated.

Profiled Normative Platform (PNP) — a composition of a governance surface, conformance workloads, and optional business domains under a conformance profile. The unit of governance and of release. Not a repository.

Conformance profile — the selection that determines which governance surface and which workloads compose a platform. Two profiles over the same material are two platforms.

Baseline (B_n) — the complete authored protocol state of a platform at a point in its history. Sealed by compilation into a snapshot.

Snapshot (S_n) — the sealed, content-identified representation of a baseline; the only artifact execution reads, and the object a pin names.

Pin — the identification of the baseline against which a transformation's claims are verified. Belongs to the change judged against it.

Determinacy condition — the property that must hold at a transition for the transformation function to be defined there.

Construction completeness — the determinacy condition at the design-to-artifact transition: whether a design uniquely determines the artifacts it schedules, measured over the shape the generator emits.

Declared emptiness — a design's explicit statement that a leaf has no value, distinguished from the absence of a statement.

Total process — the property that a transformation attempt yields exactly one of a fixed set of stated outcomes, never an undefined one.

Functioning platform — a platform satisfying the five conditions of §9: composes, seals, executes, answers, proves.

Appendix E — References

Companion papers

Ganti, B. (2026k). *Protocol-Governed Computing: An Architecture for Closed-Loop Governed Transformation*. Companion paper. DOI: <https://doi.org/10.5281/zenodo.21879948>

Ganti, B. (2026l). *Protocol-Governed Computing: An Architecture for Deterministic Declarative Execution*. Companion paper. DOI: <https://doi.org/10.5281/zenodo.21879516>

Prior work by the author

Ganti, B. (2026a). *Protocol-Governed Systems: An architectural foundation for the AI era*. Zenodo Working Paper. <https://doi.org/10.5281/zenodo.18715516>

Ganti, B. (2026b). *Protocol-Governed Systems: A Conceptual Model*. Zenodo. <https://doi.org/10.5281/zenodo.20300611>

Ganti, B. (2026c). *Protocol-Governed Systems: A constitutional realization of Turing-complete systems*. Zenodo Working Paper. <https://doi.org/10.5281/zenodo.18718409>

Ganti, B. (2026d). *The Federation-Concern Constitutional Model: A formal structural taxonomy for protocol-governed systems*. Zenodo Working Paper. <https://doi.org/10.5281/zenodo.18719589>

Ganti, B. (2026g). *Deterministic Enforcement: Runtime binding, execution, and trace conformance*. Zenodo Working Paper. <https://doi.org/10.5281/zenodo.18930314>

Ganti, B. (2026i). *The Inversion of Trust: Vocabulary-bounded security in protocol-governed systems*. Zenodo Working Paper. <https://doi.org/10.5281/zenodo.18930512>

Capability security and authority

Dennis, J. B., & Van Horn, E. C. (1966). Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3), 143–155.

Miller, M. S. (2006). *Robust composition: Towards a unified approach to access control and concurrency control*. PhD thesis, Johns Hopkins University.

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308.

Specification, verification, and models

Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 576–580.

Jackson, D. (2006). *Software Abstractions: Logic, Language, and Analysis*. MIT Press.

Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

France, R., & Rumpe, B. (2007). Model-driven development of complex software: A research roadmap. In *Future of Software Engineering (FOSE '07)*, 37–54.

Schmidt, D. C. (2006). Model-driven engineering. *IEEE Computer*, 39(2), 25–31.

Software structure and evolution

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058.

Parnas, D. L., & Clements, P. C. (1986). A rational design process: How and why to fake it. *IEEE Transactions on Software Engineering*, SE-12(2), 251–257.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9), 1060–1076.

Perry, D. E., & Wolf, A. L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4), 40–52.

Naur, P. (1985). Programming as theory building. *Microprocessing and Microprogramming*, 15(5), 253–261.

Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *IEEE Computer*, 20(4), 10–19.

Compilation, provenance, and reproducibility

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

Moreau, L., & Missier, P. (Eds.) (2013). *PROV-DM: The PROV Data Model*. W3C Recommendation.

Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70.

Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., & Cappos, J. (2019). in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, 1393–1410.

Ronsse, M., & De Bosschere, K. (1999). RecPlay: A fully integrated practical record/replay system. *ACM Transactions on Computer Systems*, 17(2), 133–152.

AI-generated code

Chen, M., et al. (2021). Evaluating large language models trained on code. *arXiv:2107.03374*.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy*, 754–768.

Author Information

Bhash Ganti (aka Bachi) — bachipeachy@gmail.com